Mini Project Report

Scientific Calculator

CS 816: Software Production Engineering

A Project Report Submitted
in Partial Fulfilment of the Requirements
for the Award of the Degree

MASTER OF TECHNOLOGY

in

Computer Science and Engineering

Submitted by

Devdeep Sarkar

(MT2025042)

Submitted to
Department of Computer Science and Engineering
International Institute of Information Technology
Bangalore - 560100, India

1. Introduction

Modern software development relies heavily on automation to enhance efficiency, reliability, and speed of application delivery. DevOps principles achieve this by seamlessly integrating development and operations, primarily through the establishment of a Continuous Integration and Continuous Deployment (CI/CD) pipeline. This pipeline is essential for ensuring that code changes are automatically tested and deployed, minimizing the need for manual intervention.

This project focuses on developing a Scientific Calculator application using Python and integrating it into a comprehensive CI/CD workflow. The application is capable of performing various mathematical operations, including square root, factorial, natural logarithm, and power calculations. The correctness of these functions is verified through unit testing.

The development workflow is automated using several key DevOps tools:

- GitHub manages version control.
- Jenkins orchestrates the entire CI/CD pipeline.
- Docker is used for containerizing the application.
- Ansible handles automated deployment.

The pipeline is triggered automatically via Webhooks and ngrok upon code pushes to the GitHub repository. Furthermore, email notifications are configured to provide users with real-time updates on the pipeline's execution status.

In conclusion, this project effectively demonstrates the integration of various DevOps tools to automate the entire software development lifecycle, leading to an efficient application deployment process.

2. Tools and Technologies Used

2.1. Application Development and Testing

- **Python:** The core language used for the Scientific Calculator application. It leverages the built-in math module for complex operations (e.g., square root, factorial, logarithm, power).
- **Unit Testing (unittest):** Python's standard unittest framework ensures the correctness and reliability of the calculator functions.

2.2. Version Control and CI/CD Automation

- **GitHub:** Serves as the version control system, hosting the source code, facilitating

collaboration, and tracking all code changes via commits.

- **Jenkins:** The central automation server for the CI/CD pipeline. It automates critical tasks, including repository cloning, project building, test execution, Docker image creation, and final application deployment.
- **GitHub Webhooks:** Trigger the Jenkins pipelines automatically, initiating a build whenever a new change is pushed to the GitHub repository.
- **Ngrok:** Creates a secure, publicly accessible URL for the local Jenkins server, enabling GitHub Webhooks to connect and trigger builds reliably.
- **Gmail SMTP:** Configures Jenkins to send email notifications, alerting users to the success or failure status of the build pipeline.

2.3. Containerization and Deployment

- **Docker:** Used to containerize the application, packaging it with all its dependencies to guarantee consistent execution across various environments.
- **Docker Hub:** A cloud-based registry for storing and sharing Docker images. Jenkins pushes the newly built image here, ready for deployment.
- **Ansible:** Automates the final deployment and configuration management tasks by pulling the latest Docker image from Docker Hub and running the application container.

3. Project Architecture Overview

The workflow of the system is:

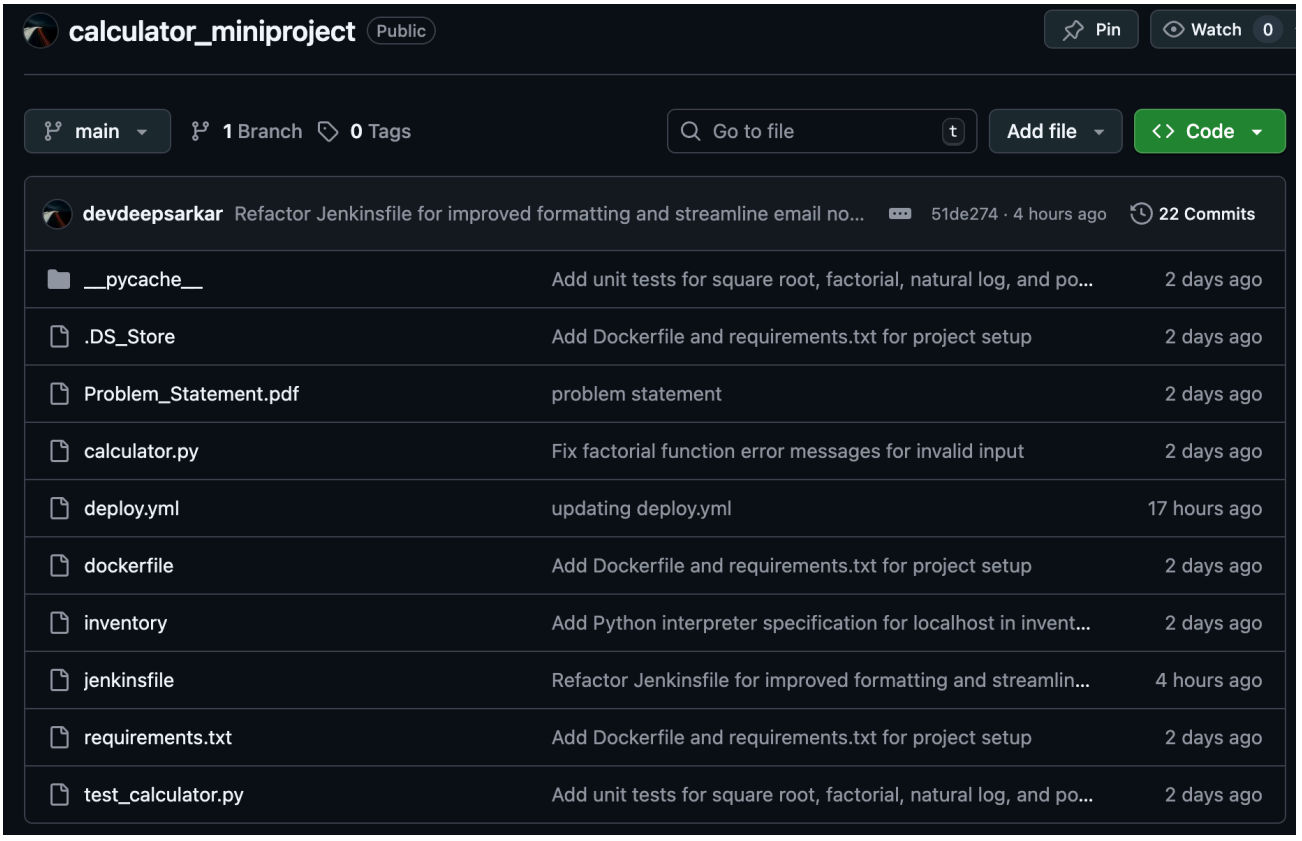
```
Developer → GitHub Repository
        ↓
Webhook Trigger
        ↓
Jenkins Pipeline
        ↓
Build and Test Application
        ↓
Docker Image Build
        ↓
Push Image to Docker Hub
        ↓
Deployment using Ansible
        ↓
Email Notification
```

4. Source Code Management (SCM)

Source Code Management is used to track changes in the project source code and maintain different versions of the application. In this project, **Git** is used as the version control system and **GitHub** is used as the remote repository to store and manage the project files. Following commands are used:

- Initialize Git Repository: *git init*
- Check Git Status: *git status*
- Add All Files to Staging Area: *git add .*
- Add Specific File to Staging Area: *git add filename*
- Commit Changes: *git commit -m "Initial project commit"*
- Add Remote GitHub Repository: *git remote add origin https://github.com/username/repository-name.git*
- Verify Remote Repository: *git remote -v*
- Push Code to GitHub: *git push -u origin main*
- Pull Latest Changes from GitHub: *git pull origin main*
- Clone GitHub Repository: *git clone https://github.com/username/repository-name.git*
- Check Git Commit History: *git log*

Following screenshot demonstrates final github repository.



5. Source Code Implementation

The source file implements a scientific calculator supporting multiple mathematical operations.

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % python3 calculator.py

Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log
4. Power
5. Exit
Enter choice: 1
Enter number: 4
2.0
```

6. Unit Testing Implementation

Unit tests verify the correctness of calculator functions. Various test cases has been implemented including edge cases like negative numbers and zero input.

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % python3 test_calculator.py
.................
----------------------------------------------------------------------
Ran 17 tests in 0.000s

OK
```

7. Docker Containerization

Docker is used to package the application into a **portable container**. The Dockerfile defines how the container image is built. Following commands is used to run and build docker images and container.

- Build Docker Image: *docker build -t devdeepsarkar/calculator .*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker build -t devdeepsarkar/calculator .
[+] Building 4.3s (10/10) FINISHED
 => [internal] load build definition from dockerfile
 => => transferring dockerfile: 149B
 => [internal] load metadata for docker.io/library/python:3.10
 => [auth] library/python:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/4] FROM docker.io/library/python:3.10@sha256:c6b2a9f40fbee6dc2ed9c2355655858ce3725c4d07aa546a102fa9ebb2d3164d
 => => resolve docker.io/library/python:3.10@sha256:c6b2a9f40fbee6dc2ed9c2355655858ce3725c4d07aa546a102fa9ebb2d3164d
 => [internal] load build context
 => => transferring context: 35.39kB
 => CACHED [2/4] WORKDIR /app
 => [3/4] COPY . /app
 => [4/4] RUN pip install -r requirements.txt
 => exporting to image
```

- Run Docker Container: *docker run -it devdeepsarkar/calculator*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker run -it devdeepsarkar/calculator

Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log
4. Power
5. Exit
Enter choice:
```

- Check Running Containers: *docker ps*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED       STATUS       PORTS                    NAMES
10078f2f65e1   de7a2e205c11   "python calculator.py"   4 hours ago   Up 4 hours   0.0.0.0:5050->5000/tcp   calculator_container
```

- View Logs: *docker logs container_name*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker logs calculator_container

Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log
4. Power
5. Exit
Enter choice: 1
Enter number: -5
Traceback (most recent call last):
  File "/app/calculator.py", line 41, in <module>
    print(square_root(x))
  File "/app/calculator.py", line 6, in square_root
    raise ValueError("Negative number not allowed")
ValueError: Negative number not allowed
```

- Stop Container: *docker stop container_name*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker stop calculator_container
calculator_container
```

- Remove Container: *docker rm container_name*

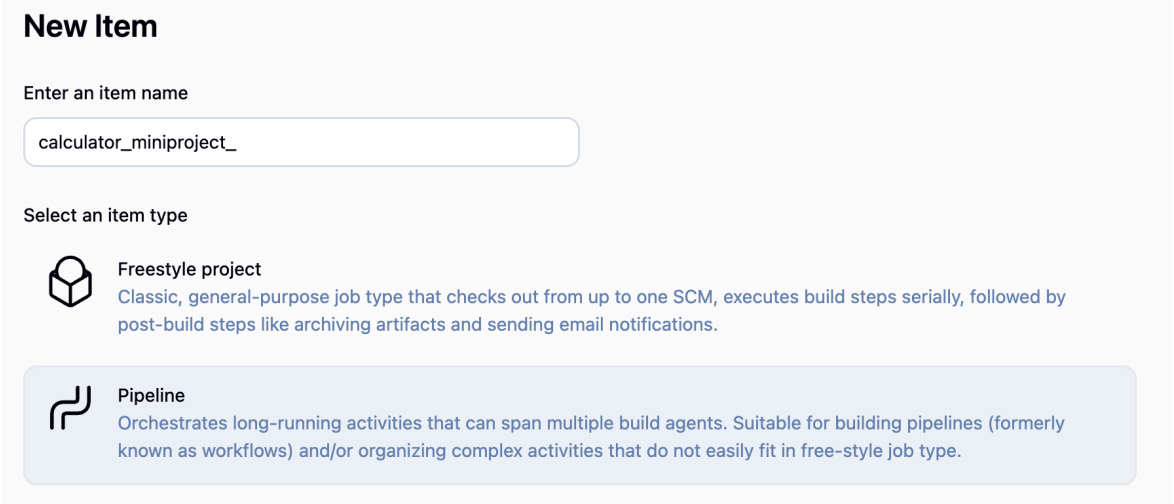
```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker rm calculator_container
calculator_container
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS_    NAMES
```

8. Jenkins Setup

Jenkins is used for Continuous Integration and Continuous Deployment (CI/CD). The jenkinsfile defines the CI/CD pipeline stages.

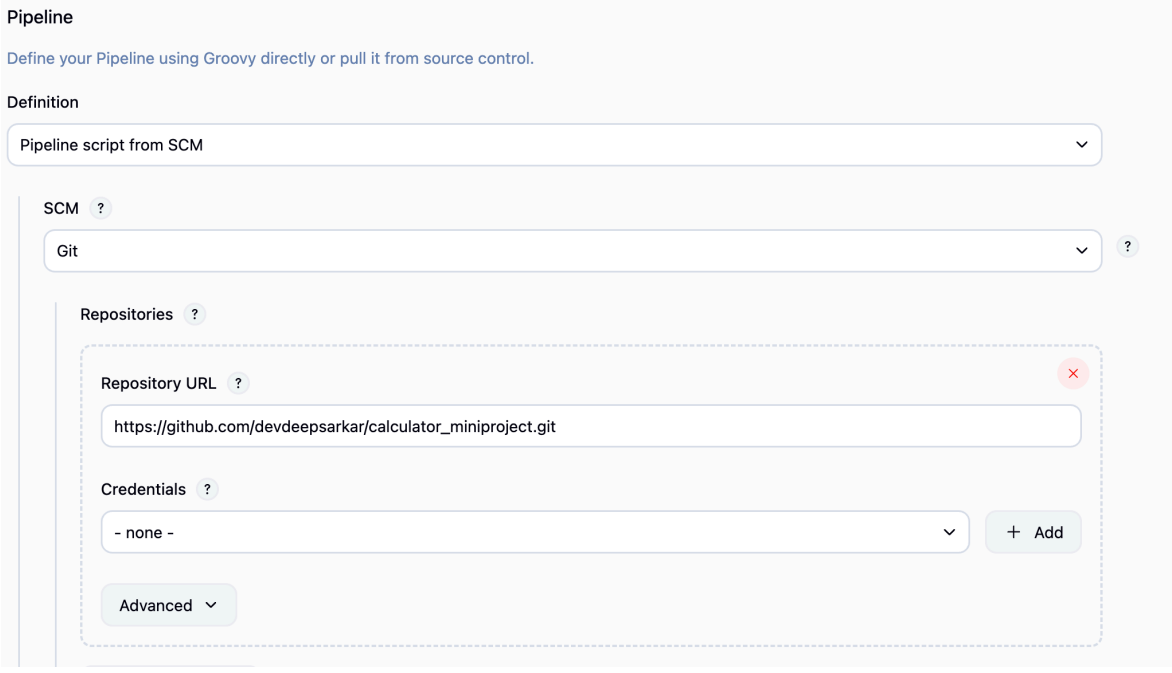
8.1. Creating Jenkins Pipeline Job

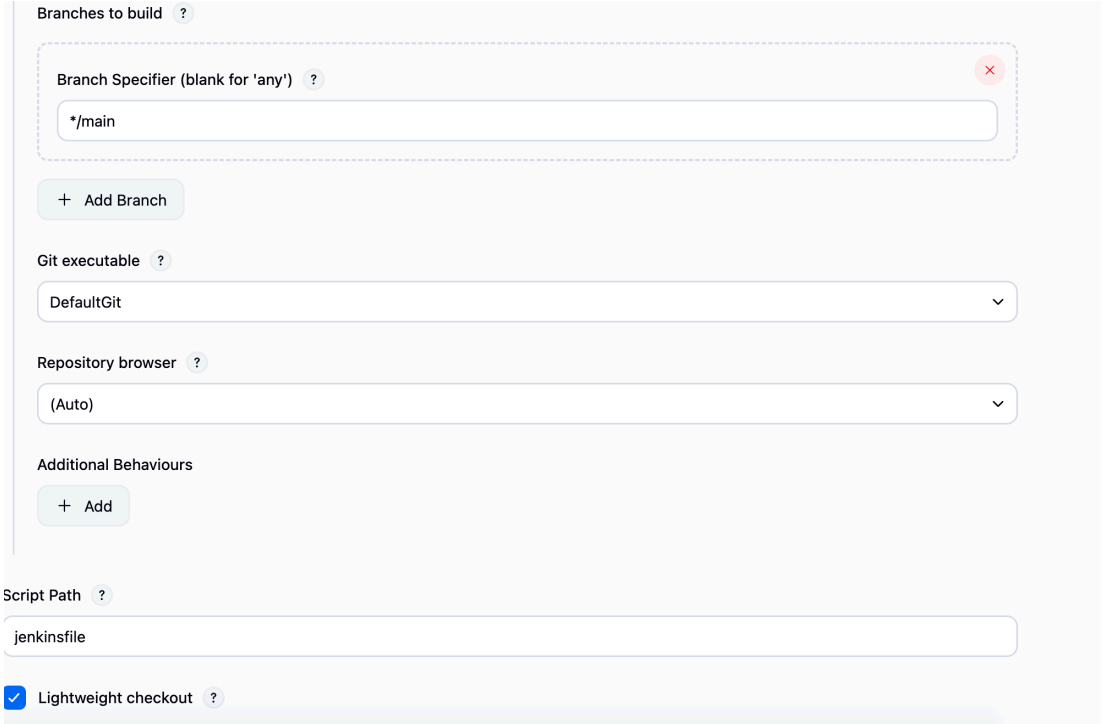
A pipeline job is created in Jenkins to define and automate the CI/CD workflow using a Jenkinsfile containing multiple build and deployment stages.



8.2. Configure Git SCM

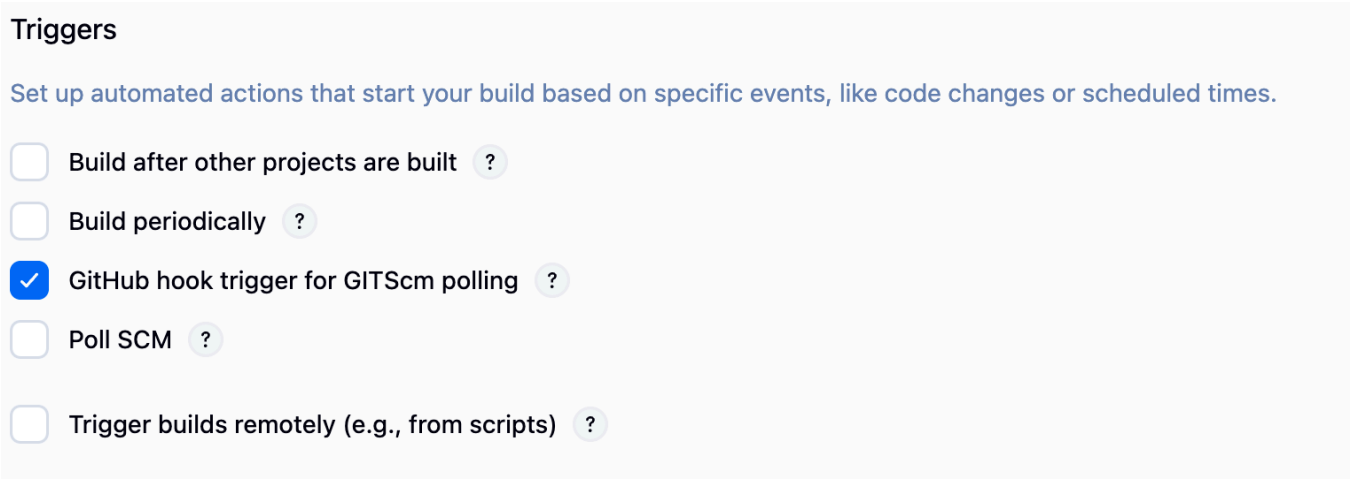
Git Source Code Management is configured in Jenkins to connect the pipeline job with the GitHub repository so that Jenkins can clone and use the project source code.





8.3. Enable Webhook Trigger

This allows Jenkins to automatically start builds when GitHub changes occur.



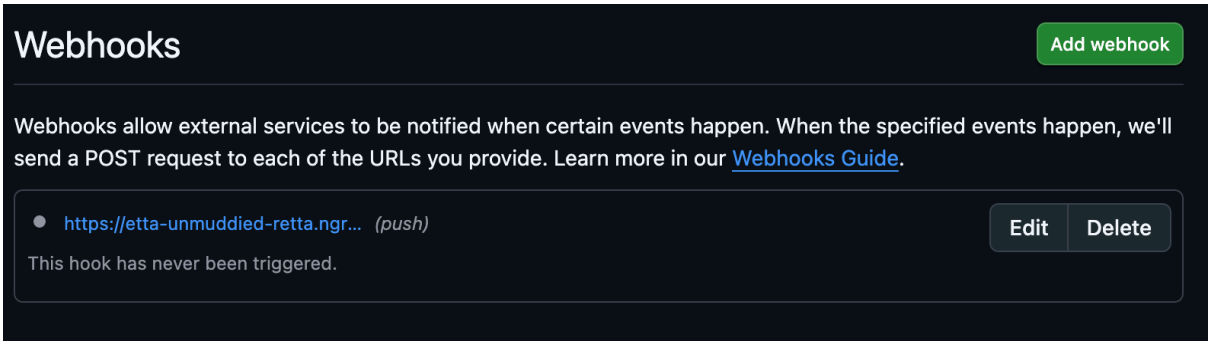
8.4. Setup ngrok

Ngrok exposes local Jenkins server to the internet.

```
ngrok                                                                          (Ctrl+C to qui

 One gateway for every AI model. Available in early access *now*: https://ngrok.com/r/ai

Session Status                online
Account                       Devdeep Sarkar (Plan: Free)
Update                        update available (version 3.36.1, Ctrl-U to update)
Version                       3.35.0
Region                        India (in)
Latency                       21ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://etta-unmuddied-retta.ngrok-free.dev -> http://localhost:8080

Connections                   ttl     opn     rt1     rt5     p50     p90
                              28      0       0.00    0.00    30.06   33.03
```

8.5. Add Webhook in GitHub

A webhook is added in the GitHub repository to automatically trigger the Jenkins pipeline whenever code changes are pushed to the repository.

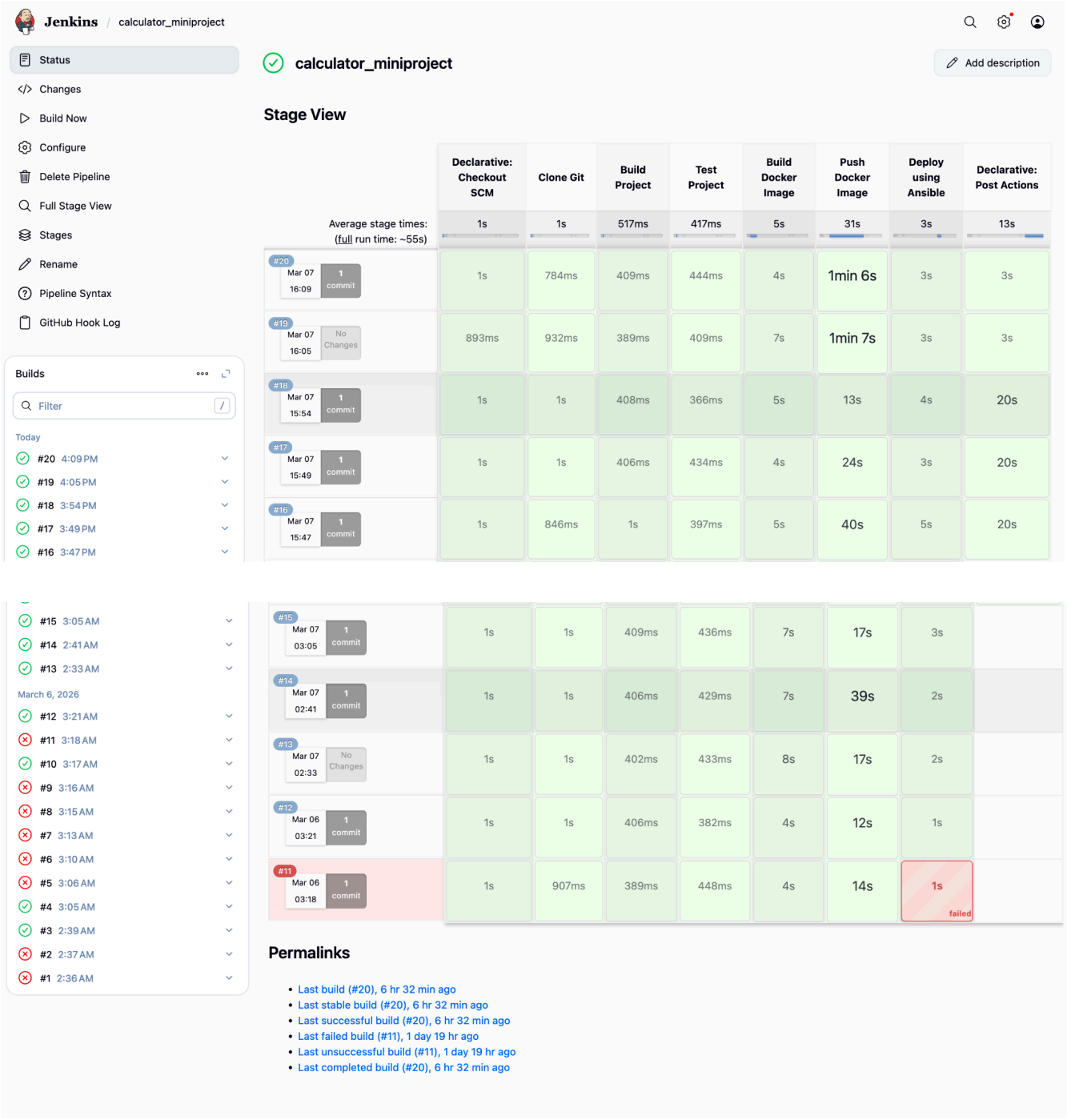


8.6. Jenkinsfile Stages

The Jenkinsfile defines the CI/CD pipeline stages. It contains the following pipeline stages.

- Clone Git: Clones repository from GitHub.
- Build Project: Installs project dependencies mentioned in requirements.txt file.
- Test Project: Runs unit tests.
- Build Docker Image: Creates Docker image.
- Push Docker Image: Pushes image to DockerHub.
- Deploy using Ansible: Runs Ansible playbook.
- Email Notification Setup: Jenkins sends email notifications after build completion.

The following figure showcases the Screenshot of all the builds and stages of jenkins.



9. Ansible Overview

Ansible is used for automated deployment and configuration management.

9.1. Inventory File

Defines the target servers. Here Deployment will occur on the local machine. Following commands are used to check & ping hosts and to test inventory file.

- Test Ansible Inventory File: *ansible-inventory -i inventory --list*

```
● devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % ansible-inventory -i inventory --list
{
    "_meta": {
        "hostvars": {
            "localhost": {
                "ansible_connection": "local",
                "ansible_python_interpreter": "/opt/homebrew/bin/python3"
            }
        },
        "profile": "inventory_legacy"
    },
    "all": {
        "children": [
            "ungrouped",
            "servers"
        ]
    },
    "servers": {
        "hosts": [
            "localhost"
        ]
    }
}
```

- Ping All Hosts in Inventory: *ansible all -i inventory -m ping*

```
● devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % ansible all -i inventory -m ping
localhost | SUCCESS => {
    "changed": false,
    "ping": "pong"
}
```

- Check Inventory Hosts: *ansible all -i inventory --list-hosts*

```
● devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % ansible all -i inventory --list-hosts
  hosts (1):
    localhost
```

9.2. deploy.yml Playbook

This file automates deployment of Docker container.

Tasks include:

- Pull latest Docker image
- Remove old container

- Start new container

The following command is used to run Ansible Playbook(deploy.yml): *ansible-playbook -i inventory deploy.yml*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % ansible-playbook -i inventory deploy.yml

PLAY [servers] ***********************************************************************************************

TASK [Gathering Facts] ***************************************************************************************
ok: [localhost]

TASK [Pull latest Docker image] ******************************************************************************
ok: [localhost]

TASK [Remove existing container if present] ******************************************************************
ok: [localhost]

TASK [Start new container] ***********************************************************************************
changed: [localhost]

PLAY RECAP ***************************************************************************************************
localhost                  : ok=4    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0
```

9.3. Deployment Verification

To verify whether the container is deployed successfully and is easily accessible.

- Check containers: *docker ps*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker ps
CONTAINER ID   IMAGE                        COMMAND                  CREATED
166ef1facf79   devdeepsarkar/calculator     "python calculator.py"   3 minutes ago
```

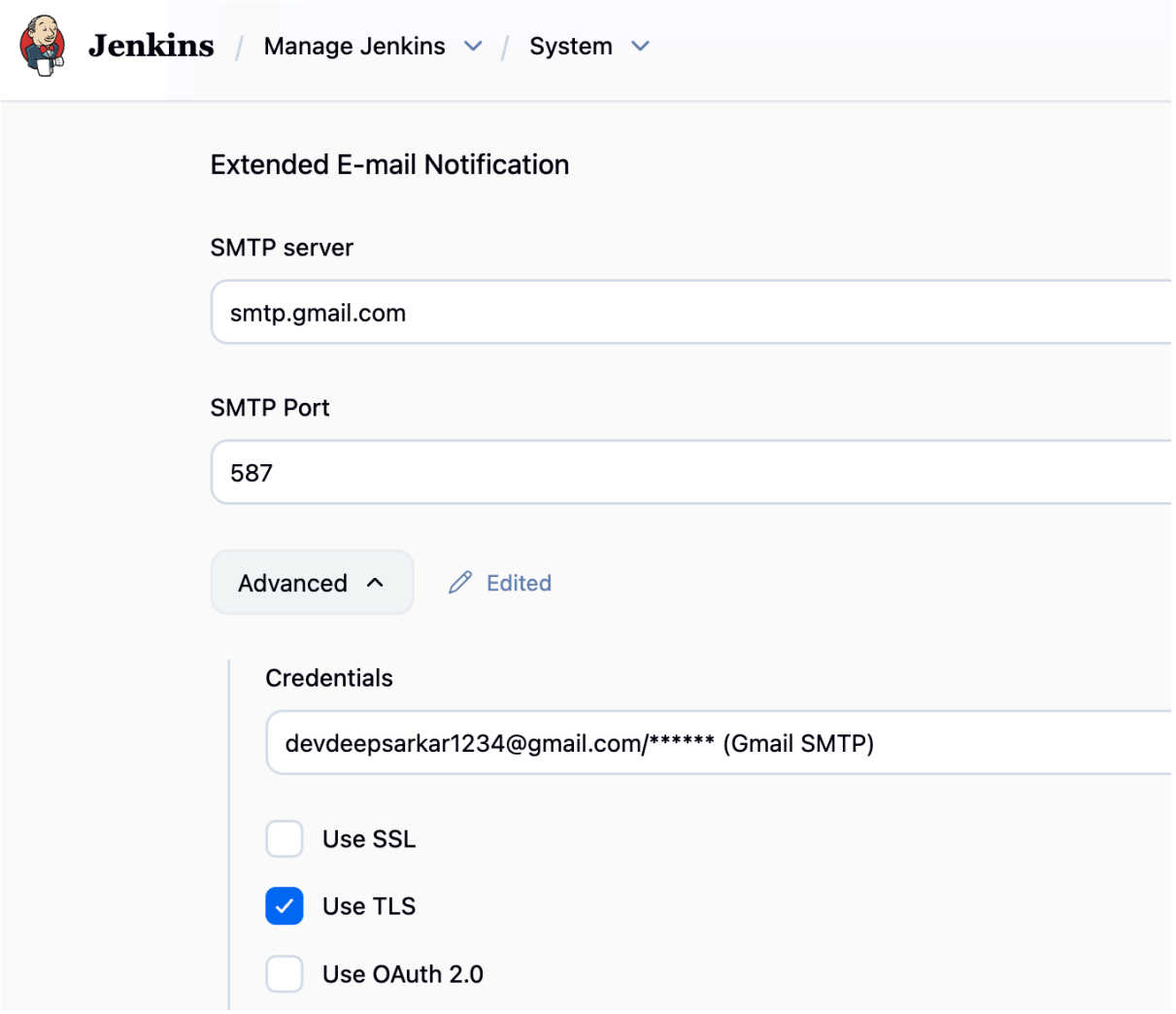
- Attach container: *docker attach calculator_container*

```
devdeepsarkar@Devdeeps-MacBook-Air calculator_miniproject % docker attach calculator_container
2
Enter number: 16
20922789888000

Scientific Calculator
1. Square Root
2. Factorial
3. Natural Log
4. Power
5. Exit
Enter choice: 5
```

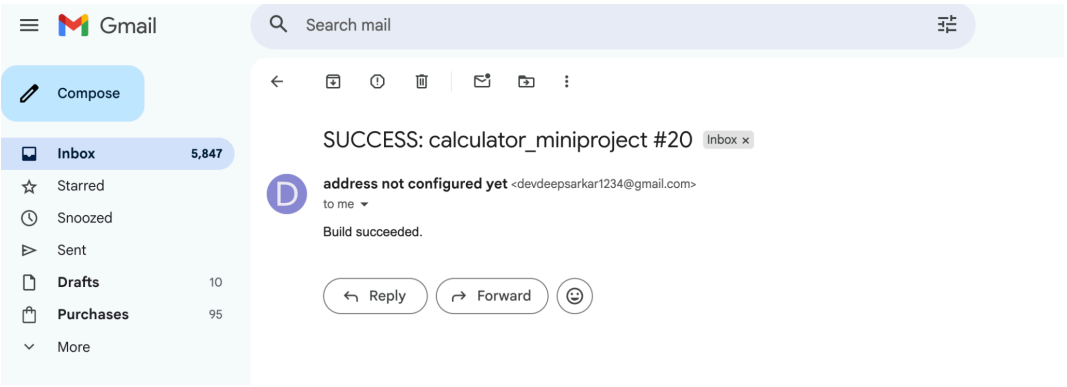
10. Email Notification Setup

Jenkins is configured with an SMTP server (Gmail SMTP) to automatically send email notifications informing users about the success or failure status of pipeline builds.

In Jenkins email notification is setup in following manner.



On successful build an email has sent to the given mail.



11. Conclusion

The project successfully demonstrates a complete CI/CD pipeline using Jenkins, Docker, GitHub, and Ansible. Automation ensures faster development cycles, reliable deployments, and efficient monitoring through notifications. This pipeline improves software delivery speed and maintains consistency across deployments.

Github Link:

https://github.com/devdeepsarkar/calculator_miniproject.git

DockerHub Link:

https://hub.docker.com/r/devdeepsarkar/calculator